

UNIVERSITY OF RUHUNA
Faculty of Engineering
Department of Computer Engineering

EC8202 Big Data Analytics

Mini Project Assessment

Hospital Patient Vital Signs Monitoring

An End-to-End Lambda Architecture Data Pipeline

Kafka · Spark Structured Streaming · Airflow · PostgreSQL

Group Members

Registration No.	Name	Role
EG/2021/4412	Arachchi W.A.T.T.W	Ingestion layer
EG/2020/3812	Akurana B.N.T.M	Processing layer
EG/2021/4686	Nadha M.J	Orchestration, storage & observability

September 2026

Contents

1	Use Case and Business Requirements	2
1.1	Scenario	2
1.2	Data sources (simulated)	2
1.3	Requirements interpreted from the use case	2
2	Architecture Decision: Lambda vs. Kappa	2
2.1	Evaluation against the use-case constraints	2
2.2	Decision and justification	2
3	System Architecture	4
3.1	Overall architecture	4
3.2	Speed layer (real-time path)	4
3.3	Batch layer and orchestration	5
3.4	Storage and serving layer	6
4	Processing Rules	7
5	Technology Stack and Justification	7
6	Observability Design	7
6.1	Structured logging (all stages)	7
6.2	Metrics	8
6.3	Alert and health-check rules	9
7	Results	9
7.1	Live ward dashboard (speed + batch views)	9
7.2	Consolidated daily risk report	10
7.3	Batch layer and orchestration	11
7.4	Failure drill: detection, diagnosis and recovery	11
7.5	Batch recomputation (“batch corrects speed”)	12
7.6	Automated tests	12
8	Limitations, Trade-offs and Production-Scale Changes	13
8.1	Limitations and trade-offs	13
8.2	What we would do differently at production scale	13
9	Conclusion	14
A	Individual Contributions	14
B	Reproducing the Results	14

1 Use Case and Business Requirements

1.1 Scenario

We implemented **Use Case 2: Hospital Patient Vital Signs Monitoring**. A hospital ward monitors patients continuously with bedside sensors. Once a day, the pathology lab uploads the day's lab results. The ward needs a platform that answers one business question:

Which patients show concerning vital-sign trends right now, and how do yesterday's lab results change the risk picture for those patients going forward?

The question has two parts with very different time scales. “*Right now*” needs answers within seconds. “*How do lab results change the picture*” is a daily reconciliation that must be complete and correct for the whole day.

1.2 Data sources (simulated)

- **Streaming source – bedside monitors** (`vitals_producer.py`). Emits one Avro-encoded reading every 2s to the Kafka topic `vitals-stream`. Each reading has `patient_id`, `heart_rate`, `spo2`, `systolic_bp`, `diastolic_bp`, `temperature`, `timestamp` and comes from a fixed pool of 15 patients (P100–P114). About 10% of readings are deliberately abnormal spikes.
- **Daily batch source – pathology lab** (`lab_results_simulator.py`). Drops one CSV file per simulated day into `lab_results_drops/`. The file holds `patient_id`, `test_type`, `result_value`, `reference_range`, `collected_at` for 5–10 randomly chosen patients and 1–3 tests each (Hemoglobin, WBC count, Creatinine, Glucose, CRP). About 15% of results fall outside their reference range.

Simulated clock. One simulated day is compressed to **120 seconds** of real time (`SIMULATED_DAY_SECONDS=120`), so a full day can be shown in a two-minute demo. The daily file dropped at time t is the lab's end-of-day extract for the simulated day $[t - 120\text{s}, t)$. The batch layer pairs it with the vitals recorded in exactly that interval.

1.3 Requirements interpreted from the use case

Table 1 lists the requirements we derived from the scenario and the brief.

2 Architecture Decision: Lambda vs. Kappa

2.1 Evaluation against the use-case constraints

The brief asks us to choose between a **Lambda** architecture (separate speed and batch paths over one immutable master dataset, merged in a serving layer) and a **Kappa** architecture (one stream-processing path; historical results come from replaying the log). Table 2 compares the two against the requirements above.

2.2 Decision and justification

We chose **Lambda**, for three reasons:

ID	Requirement
FR1	Ingest vitals continuously and lab results once per simulated day.
FR2	Clean the data (undecodable, incomplete and physiologically impossible readings) and flag abnormal readings with clinically meaningful thresholds.
FR3	Show live ward monitoring figures (active patients, abnormal rate, per-patient latest vitals) through an API endpoint and a dashboard.
FR4	Raise threshold-based alerts per patient, without duplicate alerts for one episode.
FR5	Produce a daily consolidated risk report that joins each patient’s vital-sign trend with the latest lab results, and shows explicitly how the labs change the risk.
NFR1	Latency: live figures within about 10–20 s of a reading; the daily report within one simulated day of the lab file arriving.
NFR2	Correctness and replay: the daily report must count every reading exactly once, even after crashes, restarts or late data, and any day must be recomputable.
NFR3	Observability: structured logs across all stages, metrics, and health or alert rules that detect and diagnose failures.
NFR4	Reproducibility and security: one-command start-up (Docker Compose) and no credentials in the repository.

Table 1: Functional (FR) and non-functional (NFR) requirements.

Criterion	Lambda (chosen)	Kappa (rejected)
Source shape	Fits naturally: an unbounded stream <i>and</i> a bounded daily file, each processed in its native mode.	The daily CSV must first be pushed into Kafka and joined as a stream–stream or stream–static join with long-lived state.
Latency (NFR1)	Speed layer gives seconds-level live views; the batch view is needed only once a day.	Same live latency, but the daily report would be emitted by the stream whenever state closes.
Consistency (NFR2)	The batch layer recomputes each day from the raw master dataset, so it is exact even if the speed view double-counts overlapping windows or drops late data.	Exactness depends on watermark settings. Anything later than the watermark is lost unless the whole topic is replayed.
Replay / correction	Re-run one day: an idempotent batch job replaces that day’s rows (demonstrated in Section 7.5).	Requires Kafka to retain every raw reading and a full or partial topic replay with a new job version.
Cost	The batch job runs about 7 s per simulated day; the stream keeps only minutes of state.	Needs long Kafka retention (storage cost on Confluent Cloud) and a stream holding a day of state per patient.
Complexity	Two code paths must agree on the rules (the main drawback).	One code path; this is Kappa’s main advantage.

Table 2: Lambda vs. Kappa for the ward-monitoring use case.

1. **The business question is two questions with different correctness needs.** “Right now” tolerates approximation. Our 30 s windows slide every 10 s, so every reading is counted in three windows, and a reading arriving after the watermark is ignored. The daily risk report, however, drives clinical follow-up and must count each reading exactly once. In Lambda the batch layer recomputes the day from the master dataset instead of trusting the speed view.
2. **The daily lab feed is a bounded, late-arriving dataset.** Joining it with a complete, closed day of vitals is a textbook batch computation. In Kappa, the stream would have to hold a whole day of state per patient and handle the one-file-per-day source as a stream.
3. **Cheap, targeted correction.** Because the master dataset (`vitals_readings`) is im-

mutable and keyed by Kafka coordinates, any day can be recomputed on its own. We did exactly this after a speed-layer outage (Section 7.5). Kappa would need long log retention and a replay to achieve the same.

Rejected alternative and accepted trade-off. Kappa was rejected because its main benefit, a single code path, did not outweigh the extra state, retention cost and watermark-bounded correctness for this use case. The price of Lambda is *two code paths that must agree*. We reduced this risk by keeping all thresholds in one module (`common/config.py`) and all clinical rules in one module (`processing/vitals_rules.py`). Both the Spark streaming job and the Spark batch job import these modules, and unit tests check the rules. Using Spark for both layers also means one engine, one API and one skill set.

3 System Architecture

3.1 Overall architecture

Figure 1 shows the implemented system across the ingestion, processing, storage and serving layers, with orchestration and observability across the whole pipeline.

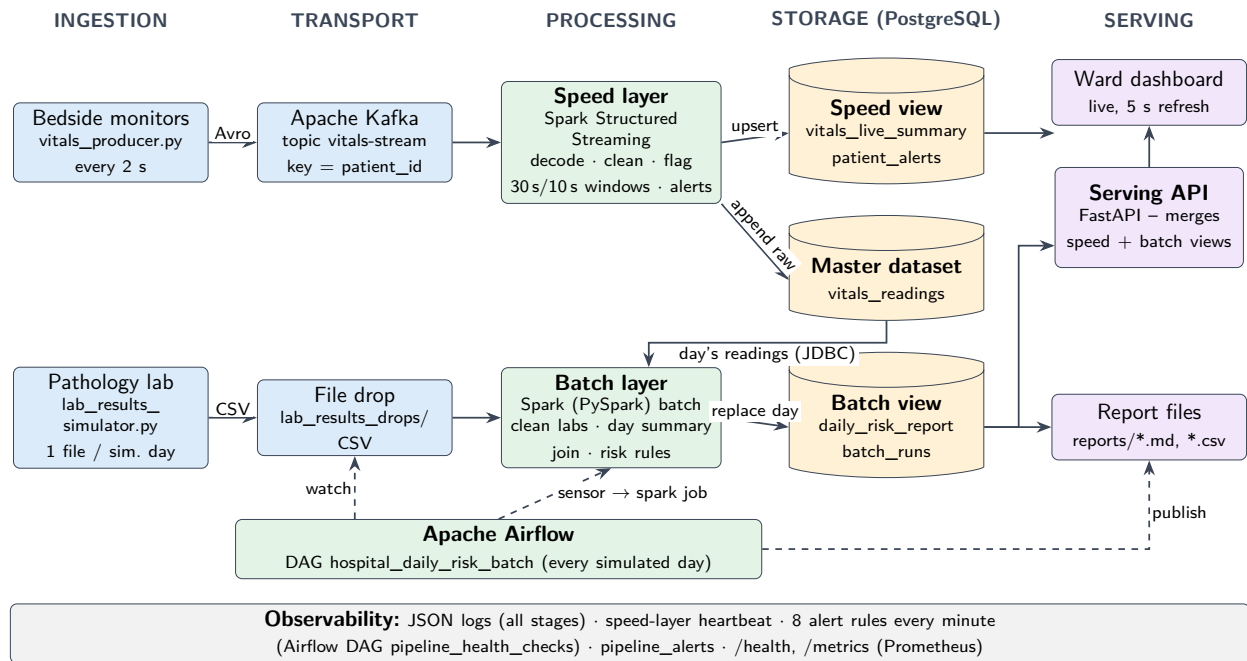


Figure 1: Implemented Lambda architecture. Solid arrows are data flow; dashed arrows are orchestration. The speed layer writes both the raw master dataset and the speed view. The batch layer recomputes each day from the master dataset.

3.2 Speed layer (real-time path)

The speed layer (`processing/speed_layer_stream.py`) is one Spark application running two streaming queries on the same Kafka source (Figure 2):

- **Decode and clean.** Payloads are decoded with `from_avro` using Member A’s schema file (`vitals.avsc`), so there is a single source of truth for the schema. Decoding runs in `PERMISSIVE` mode, so a corrupt message becomes a rejected row instead of crashing the job. Event time comes from the device timestamp, falling back to the broker timestamp. Readings

are rejected if the patient id is malformed, a value is missing, a value is physiologically impossible (e.g. HR outside 20–250 bpm, SpO₂ outside 50–100%), or systolic $\leq$ diastolic.

- **Query 1 – master dataset (append, 5 s trigger).** Every valid reading is inserted into `vitals_readings` with primary key (`kafka_partition`, `kafka_offset`) and ON CONFLICT DO NOTHING. With Spark checkpointing this gives an exactly-once *effect*: a replay after a crash cannot create duplicates.
- **Query 2 – speed view (update mode, 10 s trigger).** Event-time windows of 30 s sliding every 10 s per patient, with a 30 s watermark, compute average HR, SpO₂ and temperature, abnormal count and extremes. Windows are *upserted* into `vitals_live_summary` on (`patient_id`, `window_start`, `window_end`). Each window is also classified for a per-patient alert (Section 4). Overlapping breaching windows extend one *alert episode* in `patient_alerts`, so one spike does not produce three alerts.

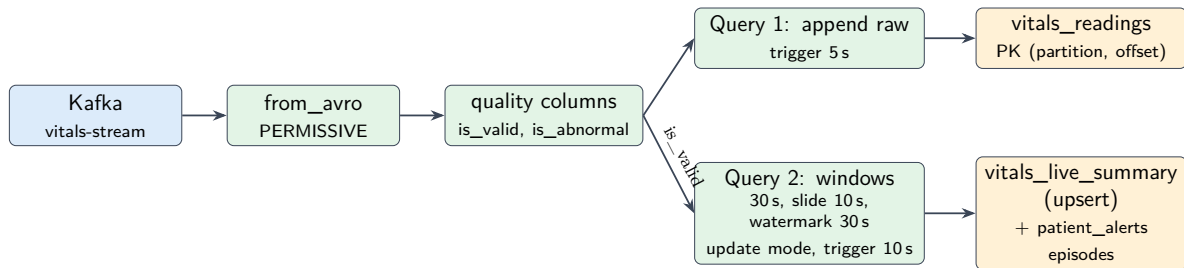


Figure 2: Speed-layer dataflow: two streaming queries share one decoded, cleaned input.

3.3 Batch layer and orchestration

The batch layer (`processing/batch_layer_daily_report.py`) runs once per lab file. It is triggered by the Airflow DAG `hospital_daily_risk_batch`, scheduled every simulated day (Figure 3):

1. **wait_for_lab_files** (`PythonSensor`) finds lab files that have no successful `batch_runs` entry and are older than a 10 s grace period, which avoids reading half-written CSVs. It then applies a **completeness gate**: a day is only processed once the master dataset holds readings past that day’s end, for example when the speed layer is still replaying Kafka after a restart. The wait is bounded; after one extra simulated day it proceeds and the job logs the window as possibly incomplete. Backlogs are handed to a single Spark run, oldest file first.
2. **run_spark_batch_job** (`BashOperator`) runs the PySpark job, passing Airflow’s `run_id` as a correlation id. The job: (a) types, validates and de-duplicates the lab CSV, parses **reference_range** with a regular expression and flags out-of-range results; (b) reads the day’s raw readings from `vitals_readings` over JDBC (the time range is pushed down to PostgreSQL); (c) summarises each patient: readings, abnormal ratio, averages, minimum SpO₂, maximum HR, and least-squares trends (**regr_slope**) of HR and SpO₂ per minute; (d) *full-outer-joins* the vitals summary with the labs on `patient_id`, so patients with vitals but no labs, and the reverse, are both reported; (e) derives the risk before and after the labs (Section 4); (f) replaces that file’s rows in `daily_risk_report` in one transaction, which makes it idempotent under Airflow retries, and records the run in `batch_runs`.
3. **validate_report_output** checks that the run succeeded and that the stored row count matches the count written.
4. **publish_report** writes the consolidated report as Markdown and CSV (`reports/daily_risk_report_dayN_<epoch>.md/.csv`).

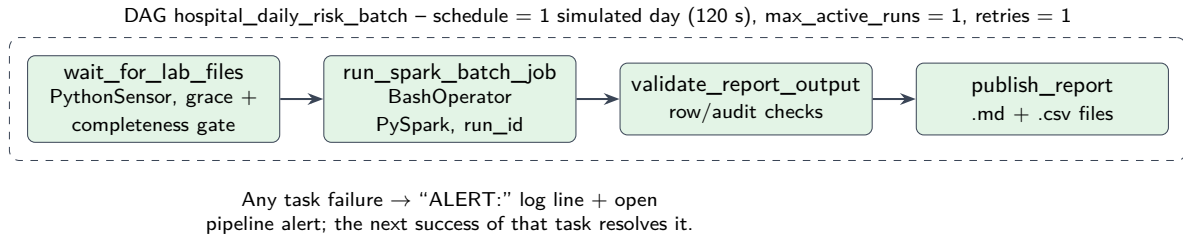


Figure 3: Airflow DAG for the batch path.

3.4 Storage and serving layer

PostgreSQL stores all views (Table 3). The schema script is additive and idempotent, so it runs safely on the team’s existing Supabase database as well as the bundled container. The FastAPI serving layer (`storage/api.py`) merges the speed and batch views. For example, `/api/ward/live` returns each patient’s latest reading, latest window and active alert next to the patient’s latest daily risk flag.

Table	Written by	Purpose / key design point
<code>vitals_readings</code>	speed layer (Q1)	Master dataset; PK (<code>kafka_partition</code> , <code>kafka_offset</code>) makes replay idempotent.
<code>vitals_live_summary</code>	speed layer (Q2)	Speed view; unique (<code>patient_id</code> , <code>window_start</code> , <code>window_end</code>) for upserts.
<code>patient_alerts</code>	speed layer (Q2)	Per-patient alert episodes (severity, reasons, extremes, window span).
<code>daily_risk_report</code>	batch layer	Batch view; one row per (patient, lab test), keyed by source file.
<code>batch_runs</code>	batch layer	Audit trail: run id, file, status, rows, patients, duration, error.
<code>pipeline_heartbeat</code>	speed-layer driver	Liveness, per-query progress and counters.
<code>pipeline_alerts</code>	health checks, Airflow	Pipeline alert lifecycle (open → resolved).

Table 3: Storage model.

Endpoint	Returns
GET <code>/api/ward/live</code>	Real-time ward figures (active patients, readings and abnormal ratio in the last 5 min, ward averages) and per-patient status.
GET <code>/api/patients/{id}</code>	Recent readings, windows, alerts and the latest daily risk for one patient.
GET <code>/api/alerts</code>	Per-patient threshold alert episodes.
GET <code>/api/reports/daily[/latest]</code>	The consolidated daily risk report(s).
GET <code>/api/pipeline/alerts, /health, /metrics</code>	Observability (Section 6).
GET <code>/</code>	Live ward dashboard (HTML, polls the API every 5 s).

Table 4: Serving API.

4 Processing Rules

Thresholds are the same ones the producer uses to label its readings, so ingestion and processing agree:

- **Abnormal reading:** $HR < 60$ or > 100 bpm, or $SpO_2 < 95\%$, or temperature $> 37.5^\circ C$.
- **Per-patient alert (per window).** *Critical* if the window average SpO_2 is below 90%, HR is below 50 or above 130 bpm, or temperature is at least $39^\circ C$. Otherwise *warning* if the window contains any abnormal reading.
- **Concerning vitals (daily).** For a patient with $n > 0$ readings, of which a are abnormal:

$$\text{concerning} \iff \frac{a}{n} \geq 0.20 \vee \overline{HR} \notin [60, 100] \vee \overline{SpO_2} < 95 \vee \overline{T} > 37.5$$

- **Abnormal lab:** the result lies outside the reference range parsed from the file.
- **Risk flag:** high (concerning vitals *and* abnormal labs), `elevated_vitals`, `elevated_labs` (vitals looked normal; the labs changed the picture) or `normal`. The report also stores `vitals_only_risk`, `risk_changed_by_labs` and a human-readable `risk_reasons`. It lists which patients were *escalated* by the labs and which were *newly flagged* by them. These two lists answer the second half of the business question directly.

5 Technology Stack and Justification

Table 5 justifies each major component against a specific constraint of the ward-monitoring scenario, rather than general popularity.

6 Observability Design

The goal is to *detect* a failure and *diagnose which stage* failed, not just to know that something is wrong.

6.1 Structured logging (all stages)

Every component logs one JSON object per line, with the same core keys as the ingestion scripts (`timestamp`, `level`, `component`, `message`) plus structured fields. Airflow's `run_id` is passed into the Spark job and stored in `batch_runs` and `daily_risk_report`, so one run can be traced from the DAG through Spark to the stored rows. Alert lines start with the marker `ALERT:`. An example from the speed layer:

```
{
  "timestamp": "2026-09-29T17:37:31.519+00:00",
  "level": "CRITICAL",
  "component": "speed_layer",
  "message": "ALERT: patient P104 critical - SpO2 89.9% < 90%; HR 160 bpm > 130; ...",
  "alert_type": "patient_vitals",
  "alert_id": 18,
  "patient_id": "P104",
  "severity": "critical"
}
```

Other log events include: rows in, written and rejected per micro-batch (with samples of rejected records); window upserts and alert counts; a heartbeat with Spark query progress (input rows/s, trigger duration, watermark, state rows); per-run batch statistics; and one access-log line per API request.

Layer	Choice	Justification tied to the use case
Ingestion	Apache Kafka (Confluent Cloud) with Avro	Bedside data must not be lost when a consumer is down. Kafka’s durable, replayable log let the speed layer catch up after our outage test with no gap. Keying by <code>patient_id</code> keeps each patient’s readings in order within a partition. Avro gives a compact, typed contract shared by the producer and Spark. A managed cluster avoided running brokers on student laptops.
Stream processing	Spark Structured Streaming	Needs event-time windows and watermarks for out-of-order sensor data, checkpointed recovery, and <code>foreachBatch</code> sinks for idempotent upserts. We chose it over Apache Storm because the same engine and DataFrame API also runs the batch layer, so the rules are shared and one skill set suffices. It also runs on Databricks, where the team first prototyped it.
Batch processing	Spark (PySpark)	Same code style and rule module as the stream. JDBC range push-down reads only one day. It scales out without a rewrite if more wards are added.
Orchestration	Apache Airflow	The daily feed needs a file sensor, ordered dependencies (sensor → Spark → validate → publish), retries and failure callbacks, plus a visible run history. Airflow also schedules the minute-by-minute health checks.
Storage	PostgreSQL (Supabase / local)	The data is small, relational and joined constantly (patients, windows, alerts, reports). <code>ON CONFLICT</code> upserts give idempotency for stream replays and batch retries, and one SQL store serves both views to the API. We preferred it to Cassandra, whose partition-oriented model suits write-heavy scale but makes the ad-hoc joins and aggregates of the serving layer awkward.
Serving	FastAPI + HTML dashboard	Lightweight JSON API matching the suggested “API endpoint for real-time ward monitoring figures”, with auto-generated OpenAPI docs and a Prometheus <code>/metrics</code> export.
Deployment	Docker Compose	One command starts PostgreSQL, the schema migration, the speed layer, the API, Airflow and optional Prometheus from a single image (Airflow + Java 17 + PySpark 3.5), giving reproducible demos.

Table 5: Technology stack.

6.2 Metrics

One module (`observability/metrics.py`) computes the metrics, which feed the rule engine, `/health` and the Prometheus `/metrics` export, so the numbers can never disagree. Table 6 lists the metrics per stage.

Stage	Metrics (Prometheus gauges, prefix <code>hospital_</code>)
Ingestion	age of newest stored reading; end-to-end event→storage lag; readings and abnormal ratio (5 min); active patients (2 min); age of newest lab file
Processing	speed-layer heartbeat age; Kafka input rows/s; reject ratio; speed-view freshness; batch successes/failures (30 min); failure ratio; last success age; last duration; pending lab files
Storage/serving	database up; active and 24 h patient alerts; open pipeline alerts; <code>health_rule_breached{rule=...}</code>

Table 6: Exported metrics.

6.3 Alert and health-check rules

Rules are stored as data (`observability/alert_rules.json`). The Airflow DAG `pipeline_health_checks` evaluates them every minute, and `observability/prometheus/alert_rules.yml` mirrors them for Prometheus. Each rule has a lifecycle: a new breach opens a `pipeline_alerts` row and logs `ALERT::`; a continuing breach only updates the row (no log spam); recovery marks the row *resolved* and logs `RESOLVED::`. While any critical rule is breached, the health task fails, so Airflow’s grid view doubles as a health history.

Rule	Condition	Severity	Diagnoses
<code>vitals_stream_stale</code>	no reading stored for 60s	critical	producer, Kafka or speed layer down
<code>speed_layer_down</code>	no heartbeat for 60s	critical	Spark job crashed (vs. producer silent)
<code>live_summary_stale</code>	no window update for 90s	warning	aggregation query stuck
<code>lab_feed_stale</code>	no lab file for 2.5 sim. days	warning	daily source late
<code>batch_backlog</code>	> 2 lab files waiting	warning	Airflow or batch falling behind
<code>batch_job_failing</code>	> 20% runs failed (30 min)	critical	batch job broken
<code>data_quality_rejects_high</code>	> 5% readings rejected	warning	producer or schema problem
<code>ward_abnormal_rate_high</code>	> 30% abnormal (5 min)	warning	ward-wide event or faulty sensors

Table 7: Pipeline alert rules. In addition, any failed Airflow task raises an alert that the task’s next success resolves.

Why these signals. Pairing *data freshness* (newest stored reading) with *process liveness* (the heartbeat) separates “the producer or Kafka is silent” from “the Spark job died”. The reject ratio separates data-quality problems from outages. The batch backlog and failure ratio localise problems to the daily path.

7 Results

Test setup. The results below come from an end-to-end run of the Docker Compose stack (speed layer, Airflow, PostgreSQL, API, Prometheus) together with Member A’s own reading generator and lab simulator (120s simulated day). For the recorded run, the producer published to a local single-broker Kafka instead of the shared Confluent Cloud cluster, and at a faster rate (one reading per second). One message in 25 was deliberately corrupted to exercise the data-quality path. This is why the dashboard shows a `data_quality_rejects_high` warning (reject ratio about 6%). Nine simulated days were processed.

7.1 Live ward dashboard (speed + batch views)

Figure 4 shows the dashboard at `http://localhost:8000`:

- **Top:** real-time ward figures (15 active patients, 295 readings and a 7% abnormal rate in the last 5 minutes, 4 patients in critical state).
- **Middle:** per-patient latest vitals, the current 30s window, active alerts and each patient’s latest daily risk flag, with recent patient-alert episodes and open pipeline alerts alongside.

- **Bottom:** the latest consolidated daily risk report.

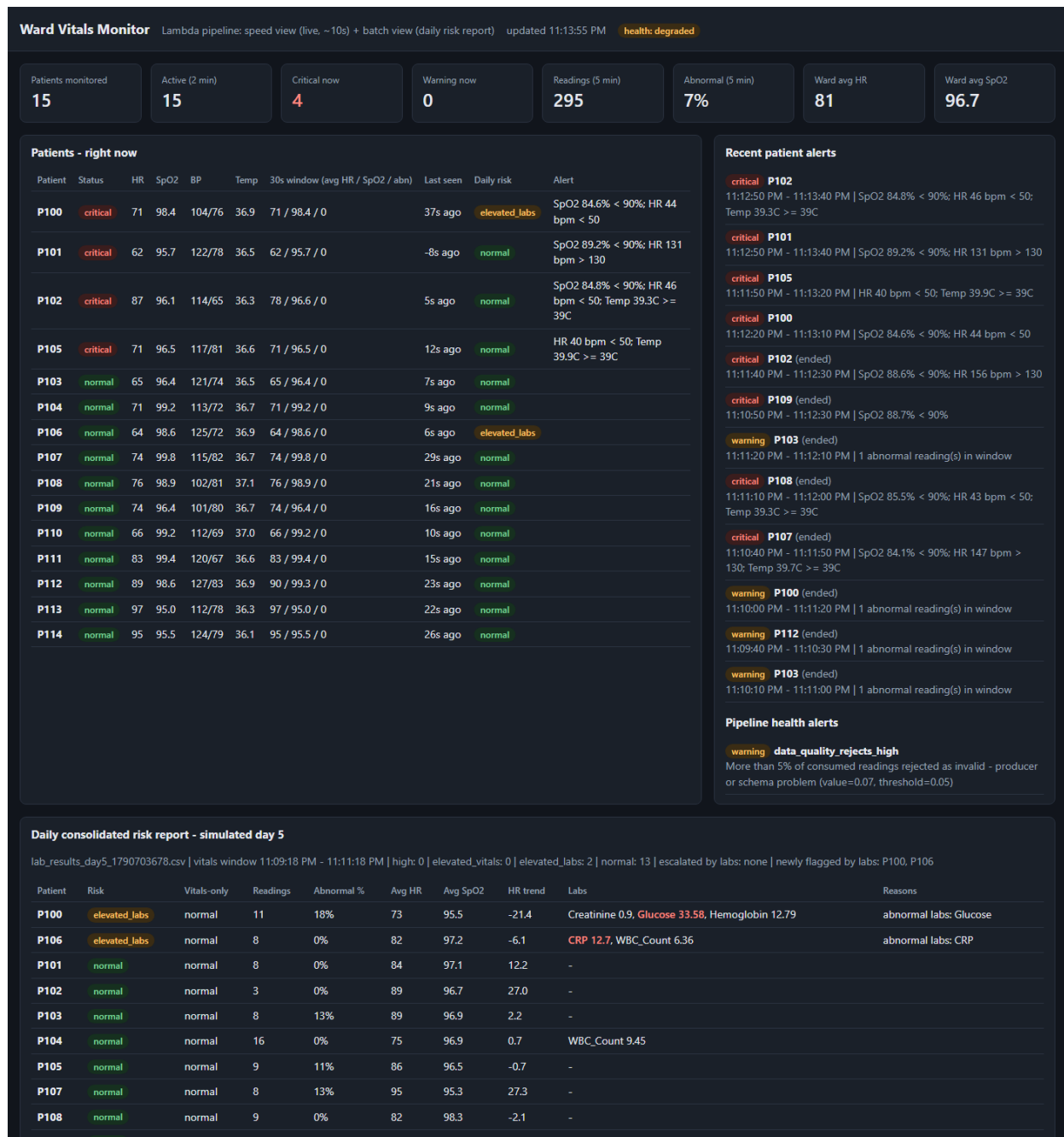


Figure 4: Live ward dashboard served by the FastAPI serving layer.

7.2 Consolidated daily risk report

Table 8 is an extract of the report generated for simulated day 3 (file `reports/daily_risk_report_day3_1790703438.md`), with 120 readings in the vitals window. Of the 15 patients: 1 *high*, 2 *elevated_vitals*, 2 *elevated_labs*, 10 *normal*. The report states directly how the labs changed the picture: **P104** was escalated from *elevated* to *high* (low average SpO₂ confirmed by an abnormal CRP), and **P100** and **P101** were *newly flagged by labs alone* although their vitals looked normal.

Patient	Risk	Vitals-only	Read.	Abn.%	Avg HR	Avg SpO ₂	Reasons
P104	high	elevated	6	17	75	94.9	avg SpO ₂ 94.9%; abnormal labs: CRP
P106	elevated_vitals	elevated	6	17	87	94.7	avg SpO ₂ 94.7%
P113	elevated_vitals	elevated	12	25	77	93.8	25% of 12 readings abnormal; avg SpO ₂ 93.8%
P100	elevated_labs	normal	10	0	75	97.9	abnormal labs: Glucose
P101	elevated_labs	normal	8	0	81	96.8	abnormal labs: Hemoglobin
P102	normal	normal	12	0	77	97.2	–
... 9 further patients classified normal							

Table 8: Extract of the consolidated daily risk report for simulated day 3.

7.3 Batch layer and orchestration

Every simulated day, Airflow detected the new lab file, ran the Spark batch job, validated the output and published the report files without manual action. Table 9 shows the first three runs as recorded in `batch_runs`. A Spark batch run takes about 6–7.5 s, including JVM start-up, which is far below the 120 s budget of a simulated day.

Lab file (simulated day)	Report rows	Patients	High risk	Spark run (s)
lab_results_day1_1790703198.csv	10	5	0	6.0
lab_results_day2_1790703318.csv	12	6	0	7.4
lab_results_day3_1790703438.csv	27	15	1	7.3

Table 9: First batch-layer runs.

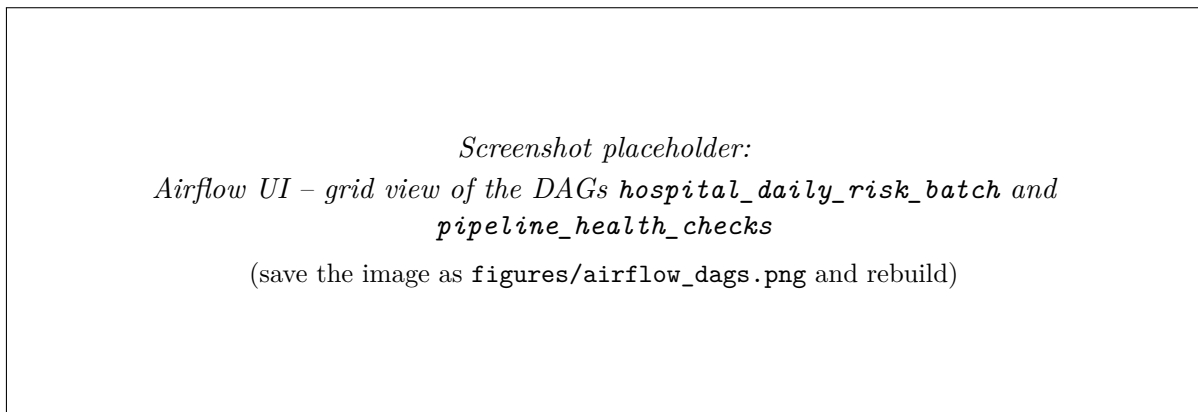


Figure 5: Airflow orchestrating the daily batch path and the minute-by-minute health checks.

7.4 Failure drill: detection, diagnosis and recovery

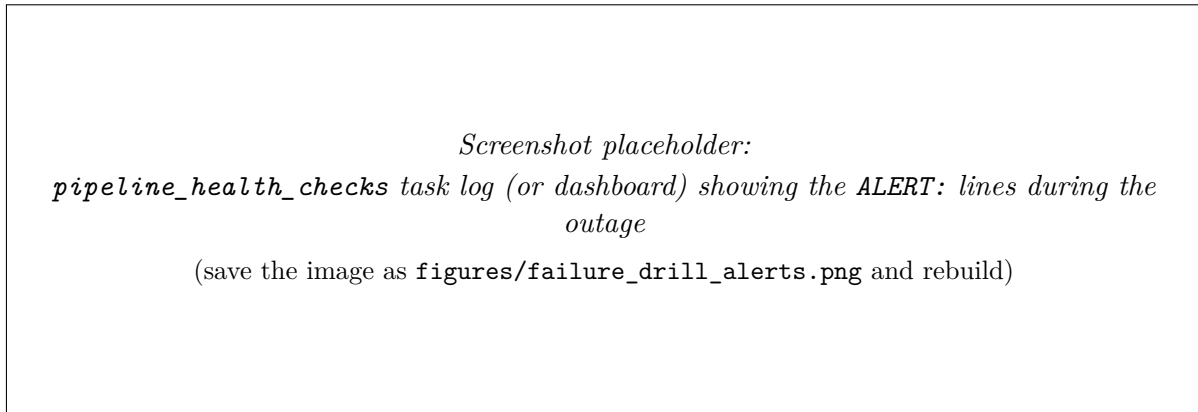
To test observability and the Lambda recovery properties, we stopped the speed layer (`docker compose stop speed-layer`) at 17:39:25 and restarted it about two minutes later.

No data loss. Counting stored readings per minute across the outage gives 58, 49, **67, 60, 57**, 58 for 17:37–17:42. The outage minutes are complete. Their readings were stored up to 2 min 38 s late, during the replay, instead of being lost. Normal end-to-end lag from event to storage was

Time (UTC)	Observed behaviour
17:39:25	Speed layer stopped; the producer keeps publishing to Kafka.
17:41:01	Health DAG opens <code>vitals_stream_stale</code> (critical), <code>speed_layer_down</code> (critical) and <code>live_summary_stale</code> (warning). ALERT: lines are logged, the health task fails (red in Airflow), the dashboard shows health: critical . The heartbeat rule identifies the Spark job, not the producer, as the cause.
≈17:41:35	Speed layer restarted; it resumes from its checkpoint and replays the missed Kafka offsets.
17:41:51	<code>vitals_stream_stale</code> and <code>live_summary_stale</code> resolved.
17:43:01	<code>speed_layer_down</code> resolved (first heartbeats observed).

Table 10: Failure-drill timeline.

about 6–18 s. The only Kafka offsets missing from `vitals_readings` are the deliberately corrupted messages, which were rejected and counted.

**Figure 6:** Alerts raised while the speed layer was stopped.

7.5 Batch recomputation (“batch corrects speed”)

The run also exposed a real completeness problem, and showed Lambda correcting it. Days 1 and 2 were computed while the speed layer was still starting up and replaying Kafka. Their reports therefore contained **0** readings, although the master dataset later held **33** and **115** readings for those windows. Two changes followed:

1. We re-ran the idempotent batch job for the two files (one `--lab-file` argument each). The rows for those days were replaced with the correct counts (33 and 115) and the other days were untouched.
2. We added the **completeness gate** described in Section 3.3, so the DAG now waits for the master dataset to cover a day before processing it. All later days matched the master dataset exactly (e.g. day 4: 106 of 106 readings; day 5: 128 of 128).

7.6 Automated tests

18 automated tests pass (`pytest`), inside the Docker image with PySpark and Java 17. They cover:

- lab-file discovery, including restarts of the simulator and the completeness gate;
- the clinical and alert rules, the health-rule engine, the Prometheus export and JSON logging;

- Spark transformations, including decoding real Avro payloads produced the same way as the producer (corrupt, impossible and bad-timestamp messages), sliding-window counts, lab cleaning and de-duplication, and the risk matrix.

8 Limitations, Trade-offs and Production-Scale Changes

8.1 Limitations and trade-offs

- **Two code paths (Lambda's cost).** The speed and batch layers must implement the same rules. This is mitigated by shared rule and threshold modules and tests, but not eliminated.
- **Sparse simulation.** 15 patients and one reading every 2s give about one reading per patient per 30s window and about 8 readings per patient per simulated day. Window averages are often a single reading, and the per-day trend slopes are noisy. The trends are reported but deliberately not used in the risk rule.
- **Driver-side writes.** Micro-batches are collected to the Spark driver and written with `psycopg2`. This is fine for one ward, but a bottleneck at scale.
- **PostgreSQL as master dataset.** It is simple and transactional but grows without bound. There is no partitioning or retention policy.
- **Batch runs locally, not on Databricks.** Databricks Community Edition has no Jobs REST API, so Airflow cannot trigger it. It also cannot read the lab files, which live on the simulator's machine. The Spark code is Databricks-compatible, but the batch job is run as local PySpark from Airflow.
- **Secrets.** Credentials come from a git-ignored `.env` file or a Databricks secret scope. Community Edition cannot create secret scopes, so notebook runs need credentials in a cell that must be cleared before export.
- **Alerting stops at logs, database and dashboard.** There is no paging integration (e.g. email, Slack, PagerDuty). Clinical thresholds are simplified population ranges, not patient-specific baselines or a validated score such as NEWS2.
- **Single-node components.** The demo runs Spark `local[2]`, Airflow's `LocalExecutor` and a single PostgreSQL instance.

8.2 What we would do differently at production scale

- Store the master dataset in a lakehouse table (Delta Lake or Apache Iceberg on object storage) partitioned by date. Keep PostgreSQL only for the small serving views, or move them to a low-latency store.
- Use distributed `foreachPartition` writers or native connectors instead of driver-side writes. Run Spark on a managed cluster (Databricks Jobs or EMR), triggered through the Airflow Databricks provider.
- Enforce data contracts with a schema registry (Confluent Schema Registry) and a dead-letter topic for rejected messages. Right-size Kafka partitions and retention per ward.
- Run Airflow with a Celery or Kubernetes executor, managed secrets (e.g. Vault or a cloud secret manager), and Grafana dashboards with Alertmanager routing to on-call staff.
- Replace the fixed thresholds with clinically validated early-warning scores and per-patient baselines, and add access control and audit logging for patient data (HIPAA- or GDPR-style compliance).

- Evaluate a Kappa-style unification once the platform runs on a lakehouse with streaming tables. Its main drawback here, reprocessing through long log retention, largely disappears when the master dataset is itself a replayable table.

9 Conclusion

We built a complete Lambda pipeline for ward vital-sign monitoring. Kafka ingests bedside readings. A Spark Structured Streaming speed layer serves live ward figures and per-patient alerts within seconds, and also persists an immutable master dataset. An Airflow-orchestrated Spark batch layer recomputes each simulated day and joins it with the pathology lab’s daily file. The resulting consolidated risk report shows, per patient, how the latest lab results escalate or newly raise the risk. The system is observable end to end: structured logs, correlated run ids, a heartbeat, exported metrics, and eight alert rules. In an outage test these detected and localised a failure within about a minute and a half, and the system then recovered with no data loss. The same test showed the main benefit of the chosen architecture: a day computed from incomplete data was corrected simply by re-running the batch layer.

A Individual Contributions

Member	Contribution
Arachchi W.A.T.T.W EG/2021/4412 (Member A – Ingestion)	Streaming vitals simulator with Avro serialisation and abnormal-spike generation (<code>vitals_producer.py</code> , <code>vitals.avsc</code>); daily lab-results simulator (<code>lab_results_simulator.py</code>); Confluent Cloud cluster and topic set-up; structured logging for ingestion; Kafka–Databricks connectivity (the <code>kafkashaded</code> . JAAS fix).
Akurana B.N.T.M EG/2020/3812 (Member B – Processing)	Spark Structured Streaming speed layer (Avro decoding, cleaning, abnormal flagging, sliding-window aggregation, master-dataset and speed-view sinks, per-patient alert episodes); Spark batch layer (lab cleaning, day recomputation from the master dataset, vitals–lab join, risk rules); Databricks prototyping; transformation tests.
Nadha M.J EG/2021/4686 (Member C – Orchestration, storage & observability)	PostgreSQL schema and migrations; FastAPI serving layer and live dashboard; Airflow DAGs (daily batch with completeness gate, health checks, failure callbacks); observability (metrics module, alert-rule engine, alert lifecycle, Prometheus export and rules); Docker Compose deployment; end-to-end and failure-drill testing.

B Reproducing the Results

```
cp .env.example .env          # add Confluent Cloud BOOTSTRAP_SERVER / API_KEY / API_SECRET
docker compose build
docker compose up -d          # add --profile monitoring for Prometheus (:9090)
python vitals_producer.py      # terminal 1 (repository root)
python lab_results_simulator.py # terminal 2 (repository root)
# Dashboard http://localhost:8000  API docs /docs  Airflow http://localhost:8080 (admin/admin)
docker compose run --rm --no-deps --entrypoint python api -m pytest -q # tests
```

Assumptions and simplifications. One simulated day is 120s. There is a fixed pool of 15 patients. Timestamps are stored in UTC. The lab file for a simulated day covers the vitals of the 120s preceding its drop time. Clinical thresholds are simplified adult reference ranges.